WEEK 2 · DAY 2 · RULE BOOK

# Does Today Remember Yesterday?

Follow it step by step. Every common error and its fix is here.

**Your goal today:** measure how much satellite data remembers its own past, find the season length from a correlogram, and catch a fake correlation. Earn **13 green ✅ ticks**. Time: about 60 minutes. Keep this page open next to Colab.

## Before you start (2 minutes)

- ☐ Open `W2D2-exercise` in Colab.
- ☐ **File → Save a copy in Drive.** Work only in the new tab.
- ☐ Run **STEP 0** (Shift + Enter). Wait for ✅ `Ready!`

## The 6 golden rules for today

| # | Rule | Why |
|---|---|---|
| 1 | **A lag is a shift.** `s.shift(1)` = yesterday, `s.shift(7)` = same weekday last week | The copy must line up with the original, row by row |
| 2 | **Autocorrelation goes on ONE column** (a Series): `s.autocorr(lag=7)` | A whole table (DataFrame) has no `.autocorr` |
| 3 | **Always `.dropna()` before `plot_acf`** | One empty value breaks the correlogram |
| 4 | **Ignore the bar at lag 0** | It is always 1: the series compared with itself |
| 5 | **The period is the GAP between spikes**, in the data's own units | Hourly data: lag 24 = one day. Daily data: lag 7 = one week |
| 6 | **Two trending series? Compare `.diff()` before you believe them** | Shared trend makes unrelated things look linked |

## The exercise, step by step

| Task | What you do | The tool you need |
|---|---|---|
| **A1** | Load throughput and look at 8 weeks | the 3-line recipe |
| **A2** | Make the "yesterday" copy | `s.shift(1)` |
| **A3** | Lag-1 autocorrelation | `s.autocorr(lag=1)` |

| Task | What you do | The tool you need |
|---|---|---|
| **A4** | Lag-7 autocorrelation | `s.autocorr(lag=7)` |
| **B1** | Draw the correlogram | `plot_acf(s.dropna(), lags=35)` |
| **B2** | Period of throughput | gap between spikes |
| **B3** | Height of the lag-0 bar | think: a series vs itself |
| **B4** | Period of NOC tickets | gap between spikes |
| **B5** | Period of hourly battery charge | careful: lags are HOURS |
| **B6** | Name the shape (season / trend / noise) | the shape table in the notebook |
| **B7** | Error of "same weekday last week" | copy the line above, change 1 to 7 |
| **C2** | Correlation of suya vs generators | `suya.corr(gens)` |
| **C3** | Correlation of their changes | `suya.diff().corr(gens.diff())` |
| **C4** | The same two tests on the amplifier data | `twta["twta_temp_c"]`, `twta["rf_drive_pct"]` |
| 🌶️ **Bonus** | Find the yearly peak lag | `acf_vals[300:].argmax() + 300` |

## Today's code, in one box

```
s = df["column"]                    # pick ONE column
s.shift(1)                          # yesterday's copy
s.autocorr(lag=7)                   # memory at lag 7, from −1 to +1

from statsmodels.graphics.tsaplots import plot_acf
plot_acf(s.dropna(), lags=35)       # the correlogram

a.corr(b)                           # two series: levels
a.diff().corr(b.diff())             # two series: changes (shared trend removed)
```

## Reading a correlogram

| You see | It means | Example today |
|---|---|---|
| Tall spikes at 7, 14, 21… | A weekly season, **period 7** | throughput, NOC tickets |
| Tall spikes at 24, 48… (hourly data) | A daily season, **period 24** | battery charge |
| High bars that fade slowly | A **trend** | transponder utilisation |
| Everything inside the shaded band | **No memory**: noise | – |

## When things go wrong: error → fix

| You see | It means | Fix |
|---|---|---|
| `AttributeError: 'DataFrame' object has no attribute 'autocorr'` | You used the whole table | Pick a column: `bw["throughput_gbps"].autocorr(lag=1)` |
| `MissingDataError` or `exog contains inf or nans` | There's a `NaN` in the series | Add `.dropna()` before `plot_acf` |
| `NameError: name 'plot_acf' is not defined` | The import cell wasn't run | Run B1 (it has the `from statsmodels…` line) |
| `NameError: name 'make_sales' is not defined` | STEP 0 wasn't run (or Colab restarted) | Run STEP 0 again |
| `KeyError: 'twta_temp_c'` | Wrong table or spelling | Run the C4 load cell; copy names from `twta.head()` |
| ❌ on B5 | You answered in days | The data is hourly: answer in **hours** |
| ❌ on the bonus (answer 65) | You forgot to add 300 back | `.argmax()` counts from the start of the slice |

## Stuck? Climb the ladder

1. **Read the ❌ hint.**
2. **Check this rule book.**
3. **Ask your neighbour.**
4. **Ask the AI button** to explain the error (code and error only, never real data).
5. **Raise your hand.**

🏁 **Finished?** Run `scoreboard()`. 13 of 13 = 🏆. Then explain to a neighbour why lag 7 beats lag 1 for throughput.